

Docker Deployment Guide

This guide provides comprehensive instructions for deploying the Classroom Participation Tracker using Docker and Docker Compose.



Prerequisites

Required Software

- **Docker:** Version 20.0+ ([Install Docker](https://docs.docker.com/get-docker/) (https://docs.docker.com/get-docker/))
- **Docker Compose:** Version 2.0+ ([Install Docker Compose](https://docs.docker.com/compose/install/) (https://docs.docker.com/compose/install/))
- **Git:** For cloning the repository

System Requirements

- **Memory:** Minimum 2GB RAM (4GB recommended)
- **Storage:** 1GB free disk space
- **Network:** Ports 3000 (app), 5432 (database), 6379 (Redis) available



Quick Start

1. Clone Repository

```
git clone https://github.com/benny2744/Classroom_Participation_Tracker_V2.git
cd classroom_participation_tracker
```

2. Fresh Start (Recommended)

```
# fresh start (wipes DB data)
docker compose down -v

# build & start
docker compose up -d --build
```

3. Verify Installation

```
# health check (local)
curl -i http://127.0.0.1:3010/api/health

# health check (remote/laptop)
curl -i http://<server-ip>:3010/api/health
```

4. Access Application

- **Application:** <http://localhost:3010> (or <http://<server-ip>:3010>)
- **Health Check:** <http://localhost:3010/api/health>
- **First Teacher Account:** Open <http://<server-ip>:3010/teacher> in the browser

Important: The app now runs on port **3010** (mapped from container port 3000). Set `NEXTAUTH_URL` in `docker-compose.yml` to the URL you actually use in the browser (IP:port or domain).
 Example: `NEXTAUTH_URL=http://<server-ip>:3010`

5. Database Management (One-liner Prisma Setup)

```
# Run inside the app container (compose will build/run it)
docker compose run --rm app sh -lc '
  export npm_config_cache=/tmp/.npm &&
  VER=$(node -p "require(\"./package.json\").dependencies["@prisma/client"] || (require(\"./package.json\").devDependencies && require(\"./package.json\").devDependencies["@prisma/client"]))) &&
  echo "Using Prisma CLI $VER" &&
  npx --yes prisma@$VER generate --schema /app/prisma/schema.prisma &&
  npx --yes prisma@$VER migrate deploy --schema /app/prisma/schema.prisma &&
  npx --yes prisma@$VER db seed --schema /app/prisma/schema.prisma || true
'
```

What's Fixed in v2.4.1

Docker Configuration Improvements

- ✓ **Removed obsolete** `version: key` from `docker-compose.yml` (deprecated in Docker Compose v2+)
- ✓ **Fixed port mapping** to 3010:3000 for external access
- ✓ **Simplified service dependencies** with proper health checks
- ✓ **Non-standalone Next.js runtime** using `next start` (more reliable)
- ✓ **Yarn 4 with node-modules linker** (avoids PnP headaches)

Prisma & Database Fixes

- ✓ **Version-matched Prisma CLI** to avoid client/CLI version skew
- ✓ **Copy source before Prisma generation** (proper build order)
- ✓ **Removed custom Prisma output paths** causing build issues
- ✓ **Simplified Prisma client generator** with standard output location
- ✓ **Stronger database health checks** with increased retries

Build Process Improvements

- ✓ **Proper multi-stage build** with optimized layer caching
- ✓ **Fixed yarn.lock compatibility** with modern Yarn version
- ✓ **Simplified Next.js config** (removed standalone complexity)
- ✓ **Better error handling** during TypeScript builds
- ✓ **Healthcheck with wget** instead of curl for lighter images

Environment & Security

- ✓ **Updated default credentials** with secure passwords
- ✓ **Proper NEXTAUTH_URL configuration** for external access
- ✓ **Simplified environment variables** (removed unused options)
- ✓ **Non-root user execution** for security

Docker File Structure

```

classroom-participation-tracker/
├── Dockerfile                # Production app container
├── Dockerfile.dev            # Development app container
├── docker-compose.yml        # Production services
├── docker-compose.dev.yml    # Development services
├── docker-entrypoint.sh      # App startup script
├── .dockerignore             # Docker ignore rules
├── .env.docker               # Environment template
├── .env.docker.local         # Local environment (create this)
├── database/
│   └── init/
│       └── 01-init.sql       # Database initialization
├── docker-scripts/
│   ├── setup.sh              # Initial setup
│   ├── backup.sh             # Database backup
│   ├── restore.sh            # Database restore
│   └── wait-for-db.sh        # Database health check

```

Environment Configuration

Required Environment Variables

Variable	Description	Default	Required
<code>DATABASE_URL</code>	PostgreSQL connection string	Generated	Yes
<code>NEXTAUTH_URL</code>	Application base URL	<code>http://localhost:3000</code>	Yes
<code>NEXTAUTH_SECRET</code>	Authentication secret key	Generated	Yes
<code>POSTGRES_DB</code>	Database name	<code>participation_tracker</code>	Yes
<code>POSTGRES_USER</code>	Database user	<code>tracker_user</code>	Yes
<code>POSTGRES_PASSWORD</code>	Database password	Generated	Yes

Optional Environment Variables

Variable	Description	Default
NODE_ENV	Application environment	production
PORT	Application port	3000
REDIS_URL	Redis connection string	redis://redis:6379
AWS_BUCKET_NAME	S3 bucket for file uploads	-
AWS_FOLDER_PREFIX	S3 folder prefix	participation-tracker/

Sample .env.docker.local

```
# Database Configuration
DATABASE_URL=postgresql://tracker_user:your_secure_password@database:5432/participation_tracker?connect_timeout=15

# Authentication Configuration
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=your-very-secure-secret-key-here

# PostgreSQL Settings
POSTGRES_DB=participation_tracker
POSTGRES_USER=tracker_user
POSTGRES_PASSWORD=your_secure_password

# Optional: Redis
REDIS_URL=redis://redis:6379

# Application
NODE_ENV=production
PORT=3000
```

Docker Commands

Basic Operations

```
# Start services
docker-compose up -d

# Stop services
docker-compose down

# Restart services
docker-compose restart

# View logs
docker-compose logs -f [service_name]

# Check service status
docker-compose ps
```

Development Mode

```
# Start development environment with hot reload
docker-compose -f docker-compose.dev.yml up --build

# Access development app at http://localhost:3001
```

Database Operations

```
# Access PostgreSQL shell
docker-compose exec database psql -U tracker_user -d participation_tracker

# Create database backup
./docker-scripts/backup.sh

# Restore from backup
./docker-scripts/restore.sh TIMESTAMP

# Reset database
docker-compose exec database psql -U tracker_user -d participation_tracker -c "DROP
SCHEMA public CASCADE; CREATE SCHEMA public;"
```

Application Management

```
# Rebuild application only
docker-compose build app
docker-compose up -d app

# Run Prisma commands
docker-compose exec app npx prisma db push
docker-compose exec app npx prisma db seed
docker-compose exec app npx prisma studio

# Access application shell
docker-compose exec app sh
```



Service Architecture

Services Overview

1. Application Service (app)

- **Image:** Custom Next.js build
- **Port:** 3000
- **Features:** Standalone Next.js application with Prisma ORM
- **Health Check:** GET /api/health
- **Dependencies:** Database service

2. Database Service (database)

- **Image:** postgres:15-alpine
- **Port:** 5432
- **Features:** PostgreSQL with automatic initialization
- **Health Check:** pg_isready command
- **Persistence:** Named volume (postgres_data)

3. Redis Service (redis) - Optional

- **Image:** redis:7-alpine
- **Port:** 6379
- **Features:** Session storage and caching
- **Health Check:** Redis ping command
- **Persistence:** Named volume (redis_data)

Docker Networks

- **participation_network:** Bridge network for service communication

Docker Volumes

- **postgres_data:** PostgreSQL data persistence
- **redis_data:** Redis data persistence
- **uploads_data:** Application file uploads

Security Considerations

Production Security

1. **Environment Variables:** Use strong passwords and secure secrets
2. **Network Security:** Configure firewall rules for exposed ports
3. **Database Security:** Use non-default credentials and enable SSL
4. **Container Security:** Regular image updates and vulnerability scanning

Security Best Practices

```
# Generate secure secrets
openssl rand -base64 32 # For NEXTAUTH_SECRET
openssl rand -base64 16 # For database password

# Use Docker secrets in production
docker secret create db_password password.txt
```

Performance Optimization

Production Optimizations

1. **Resource Limits:** Set memory and CPU limits in docker-compose.yml
2. **Caching:** Enable Redis for session storage
3. **Database:** Configure PostgreSQL for production workloads
4. **Monitoring:** Use Docker health checks and monitoring tools

Example Resource Configuration

```
services:
  app:
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 1G
        reservations:
          cpus: '1.0'
          memory: 512M
```

Monitoring and Logging

Health Monitoring

```
# Check all service health
docker-compose ps

# Application health check
curl http://localhost:3000/api/health

# Database health check
docker-compose exec database pg_isready -U tracker_user
```

Log Management

```
# View application logs
docker-compose logs -f app

# View database logs
docker-compose logs -f database

# View all logs
docker-compose logs -f

# Log rotation (production)
docker-compose logs --tail=100 -f
```

Backup and Restore

Automated Backup

```
# Create full backup
./docker-scripts/backup.sh

# Schedule daily backups (crontab)
0 2 * * * /path/to/classroom-participation-tracker/docker-scripts/backup.sh
```

Manual Backup

```
# Database only
docker-compose exec database pg_dump -U tracker_user participation_tracker > backup.sql

# Files and configuration
tar -czf backup.tar.gz uploads/ .env.docker.local
```

Restore Process

```
# Restore from automated backup
./docker-scripts/restore.sh 20240919_143000

# Manual restore
docker-compose exec -T database psql -U tracker_user -d participation_tracker < backup.sql
```

Troubleshooting

Issues Fixed in v2.4.0

✓ SOLVED: Docker Build Failures

Previous Error:

```
ERROR: failed to solve: failed to compute cache key: "/app/yarn.lock": not found
```

Solution: Automatic conversion of yarn.lock symlink to real file during setup.

Previous Error:

```
SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data
```

Solution: Removed build-time secrets, moved to runtime environment only.

Previous Error:

```
LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy format
```

Solution: Updated all ENV statements to modern key=value format.

Common Issues & Solutions

Application Won't Start

```
# Step 1: Check all service logs
docker compose logs -f

# Step 2: Verify database connectivity
docker compose exec database pg_isready -U tracker_user

# Step 3: Check application logs specifically
docker compose logs -f app

# Step 4: Verify environment variables
docker compose exec app env | grep -E "(DATABASE_URL|NEXTAUTH)"

# Step 5: Force rebuild if needed
docker compose down
docker compose up --build --force-recreate -d
```

Database Connection Issues

```
# Check database service status
docker compose ps database

# Test database connectivity from app container
docker compose exec app npx prisma db push

# Verify connection string format
docker compose exec app echo $DATABASE_URL

# Reset database if needed
docker compose exec database psql -U tracker_user -d participation_tracker -c "DROP
SCHEMA public CASCADE; CREATE SCHEMA public;"
```

Port Conflicts (Common on Linux/WSL)

```
# Check what's using port 3000
sudo netstat -tulpn | grep :3000
# or
sudo lsof -i :3000

# Stop conflicting services
sudo systemctl stop nginx # if nginx is running
sudo pkill -f "node.*3000" # kill any node processes on port 3000

# Use different ports by modifying docker-compose.yml
# Change "3000:3000" to "3001:3000" in the ports section
```

Docker Installation Issues (Ubuntu/Debian)

```
# If Docker is not found after installation
sudo systemctl start docker
sudo systemctl enable docker

# If permission denied errors
sudo usermod -aG docker $USER
# Then logout and login again

# If old Docker Compose installed
sudo apt remove docker-compose
# Use built-in compose plugin: docker compose (not docker-compose)
```

Permission Issues

```
# Fix script permissions
chmod +x docker-entrypoint.sh
chmod +x docker-scripts/*.sh

# Fix yarn.lock if it's still a symlink
cd app
if [ -L "yarn.lock" ]; then
    YARN_TARGET=$(readlink yarn.lock)
    cp "$YARN_TARGET" yarn.lock.temp
    rm yarn.lock
    mv yarn.lock.temp yarn.lock
fi
```

Build Context Issues

```
# Clear Docker build cache
docker builder prune -f
docker system prune -af

# Rebuild from scratch
docker compose down --rmi all --volumes --remove-orphans
docker compose up --build -d
```

Health Check Failures

```
# Check health check endpoint manually
curl -v http://localhost:3000/api/health

# If 404 error, the app might not be built with standalone output
docker compose exec app ls -la .next/
docker compose exec app ls -la server.js

# Check health check logs
docker compose exec app curl -f localhost:3000/api/health || echo "Health check failed"
```

Debug Commands

Container Inspection

```
# Interactive shell in app container
docker compose exec app sh

# Check file structure
docker compose exec app ls -la
docker compose exec app ls -la .next/

# Check environment variables
docker compose exec app env

# Check process status
docker compose exec app ps aux
```

Network Debugging

```
# Check Docker networks
docker network ls
docker network inspect classroom-participation-tracker_participation_network

# Test service-to-service connectivity
docker compose exec app ping database
docker compose exec app nc -zv database 5432
```

Database Debugging

```
# Access PostgreSQL directly
docker compose exec database psql -U tracker_user -d participation_tracker

# Check database tables
docker compose exec database psql -U tracker_user -d participation_tracker -c "\dt"

# Check database connection from app
docker compose exec app npx prisma studio --browser none
```

Performance Monitoring

```
# Monitor resource usage
docker stats

# Check logs with timestamps
docker compose logs -f -t

# Monitor specific service
docker compose logs -f app
```

Advanced Troubleshooting

Complete Clean Rebuild

```
# Stop all services
docker compose down --volumes --remove-orphans

# Remove all images
docker rmi $(docker images classroom-participation-tracker* -q) 2>/dev/null || true

# Clean build cache
docker builder prune -af

# Clean system
docker system prune -af

# Rebuild everything
./docker-scripts/setup.sh
docker compose up --build -d
```

Backup Before Troubleshooting

```
# Create backup
./docker-scripts/backup.sh

# This creates a backup in the format: backup_YYYYMMDD_HHMMSS.tar.gz
# Contains database dump and configuration files
```

Updates and Maintenance

Application Updates

```
# Pull latest changes
git pull origin main

# Rebuild and restart
docker-compose down
docker-compose up --build -d

# Run database migrations
docker-compose exec app npx prisma db push
```

System Maintenance

```
# Clean unused Docker resources
docker system prune -f

# Update Docker images
docker-compose pull
docker-compose up -d

# Monitor disk usage
docker system df
```

Version Management

```
# Tag current deployment
docker tag classroom-participation-tracker_app:latest classroom-participation-tracker_app:v2.3.1

# Rollback if needed
docker-compose down
docker tag classroom-participation-tracker_app:v2.3.0 classroom-participation-tracker_app:latest
docker-compose up -d
```

Support

Getting Help

1. **Check Logs:** Always start with `docker-compose logs -f`
2. **Health Checks:** Verify service status with health endpoints
3. **Documentation:** Reference this guide and the main README.md
4. **Community:** Submit issues on GitHub with full log output

Useful Links

- [Docker Documentation](https://docs.docker.com/) (https://docs.docker.com/)
- [Docker Compose Reference](https://docs.docker.com/compose/) (https://docs.docker.com/compose/)
- [Next.js Docker Guide](https://nextjs.org/docs/deployment#docker-image) (https://nextjs.org/docs/deployment#docker-image)
- [PostgreSQL Docker Guide](https://hub.docker.com/_/postgres) (https://hub.docker.com/_/postgres)

Version: 2.3.1 (Docker)

Last Updated: September 2024

Docker Compose: v2.0+

Supported Platforms: Linux, macOS, Windows (WSL2)